TensorFlow is loaded with computational as well as basic features like:

- Constants
- Variables
- Placeholders

Constants: The value of constants does not change. For example, in the 'Hello World' code we have initialised two constants and the value is fixed.

Variables: These are objects in TensorFlow, of which the value will be 're-filled' every time they are called or run in a session. These variables will maintain a fixed state in a graph. The *tf.Variable()* is used to declare or call variables in TensorFlow.

Placeholders: These are built-in structures for feeding input data. Sometimes these are thought of as empty variables that will be filled with data in the future. Placeholders have an optional 'shape' argument. The default option is *None*; otherwise, it will be assumed to be of any size.

Sessions in TensorFlow: Any tensor must reside in the computer memory in order to be useful to computer programmers. Once this interactive session is loaded, we are good to program. *tf.InteractiveSession()* is used to start the session and until we close or come out of this window, this session will be live.

Figures 3, 4, 5 and 6 show the usage of constant, *Hello World* of TensorFlow, variables and placeholders with examples in the Jupyter notebook.

TensorFlow graphs

TensorFlow has inbuilt features that help us to build algorithms, and computing operations that assist us to interact with one another. These interactions are nothing but graphs, also called computational graphs.

TensorFlow optimises the computations with the help of the graphs' connectivity. Each of these graphs has its own set of nodes and dependencies. Each of the types in TensorFlow — like constants, variables and placeholders — creates graphs for connectivity and computation.

Here is a coding example of linear regression using random variable generation:

```
import tensorflow as tf
print(tf.__version__)
import numpy as np
import matplotlib.pyplot as plt

%config InlineBackend.figure_format = 'svg'

C:\Users\Sony\Anaconda3\lib\site-packages\h5py\__init__.
py:34: FutureWarning: Conversion of the second argument of
issubdtype from `float` to `np.floating` is deprecated. In
future, it will be treated as `np.float64 == np.dtype(float).
type`.

from ._conv import register_converters as _register_
converters

1.10.0

# Below are hyper parameters like learning rate, number of
```

```
In [3]: import tensorflow as tf
        tf.InteractiveSession()
Out[3]: <tensorflow.python.client.session.InteractiveSession at 0x1e4752552b0>

        Constant Tensors

In [4]: tf.zeros(5)
Out[4]: <tf.Tensor 'zeros_1:0' shape=(5,) dtype=float32>

In [0]: a_const = tf.zeros(5)
        a_const.eval()
Out[6]: array([0., 0., 0., 0., 0.], dtype=float32)

In [7]: b_const = tf.zeros((2,4))
        b_const.eval()
Out[7]: array([[0., 0., 0., 0.],
               [0., 0., 0., 0.]], dtype=float32)

In [9]: c_const = tf.ones((2,3))
        c_const.eval()
Out[9]: array([[1., 1., 1.],
               [1., 1., 1.]], dtype=float32)
```

Figure 3: TensorFlow constants

```
Out[10]: 3

In [11]: import tensorflow as tf

In [12]: hello = tf.constant("Hello")
         world = tf.constant("World")
         helloworld = hello + world

In [13]: helloworld.eval()
Out[13]: b'HelloWorld'
```

Figure 4: TensorFlow *HelloWorld*

```
In [14]: val = tf.random_normal((1,5), 0, 1)

In [15]: my_var = tf.Variable(val, name='my_var')

In [19]: init=tf.global_variables_initializer()

In [25]: print("Before run: \n{}".format(my_var))
         Before run:
         <tf.Variable 'var:0' shape=(1, 5) dtype=float32_ref>

In [24]: with tf.Session() as sess:
             sess.run(init)
             after_var = sess.run(my_var)

         print("\nAfter run:\n{}".format(after_var))


         After run:
         [[ 0.23448779  0.57947916  1.1972933   1.4584718  -0.08119686]]
```

Figure 5: TensorFlow variables

```
In [28]: import numpy as np
         x_data = np.random.randn(5,10)
         y_data = np.random.randn(10,1)

In [30]: with tf.Graph().as_default():
             x = tf.placeholder(tf.float32, shape=(5,10))
             w = tf.placeholder(tf.float32, shape=(10,1))
             b = tf.fill((5,1), -1.)
             xw = tf.matmul(x, w)

             xwb = xw + b
             s = tf.reduce_max(xwb)

             with tf.Session() as sess:
                 outs = sess.run(s, feed_dict = {x: x_data, w:y_data})

         print("outs = {}".format(outs))
         outs = 1.6235082149505615
```

Figure 6: TensorFlow placeholders